Static Analysis of Serverless Applications for Security

PhD Candidate: Giuseppe Raffa

Supervisory Team: J. Blasco, D. O'Keeffe, S. K. Dash

PhD Viva Presentation - 17th September 2025

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Motivation & Challenges

- **Why serverless computing?**
  - No infrastructure management
- **Why static analysis?**
  - Most serverless security tools rely on dynamic analysis
  - Static analysis => Inspect application before deployment
  - Static analysis => Effective supplement
- **What are the challenges?**

Event-driven applications	Mechanisms to define permissions	Platform services black-box nature

Research Questions

RQ1 → **What are the services and APIs most frequently used in real-world serverless applications?**

RQ2 → **How can we design security-oriented serverless microbenchmarks?**

RQ3 → **How can a serverless-specific static analysis pipeline be structured?**

RQ4 → **Are real-world serverless applications affected by vulnerabilities that can be identified statically?**

Contributions

RQ1 →	**AWSomePy dataset characterization**	**Ch. 3**
RQ2 →	**CloudBench microbenchmark suite**	**Ch. 4**
RQ3 →	**CloudFlow framework**	**Ch. 5**
RQ4 →	**Analysis of the AWSomePy dataset**	**Ch. 6**

AWSomePy Characterization (1)

- **Cloud services**
  - Client or resource objects
  - 46 services in total

boto3.client('s3')

boto3.resource('s3')

Services	No. of Repositories	Occurrences
s3	59	217
dynamodb	47	201
lambda	24	47
ssm	14	46
sqs	21	41
sns	11	30
ec2	12	29
sts	9	26
rekognition	8	15
cloudformation	7	14
stepfunctions	9	14

Data storage and NoSQL services the most common

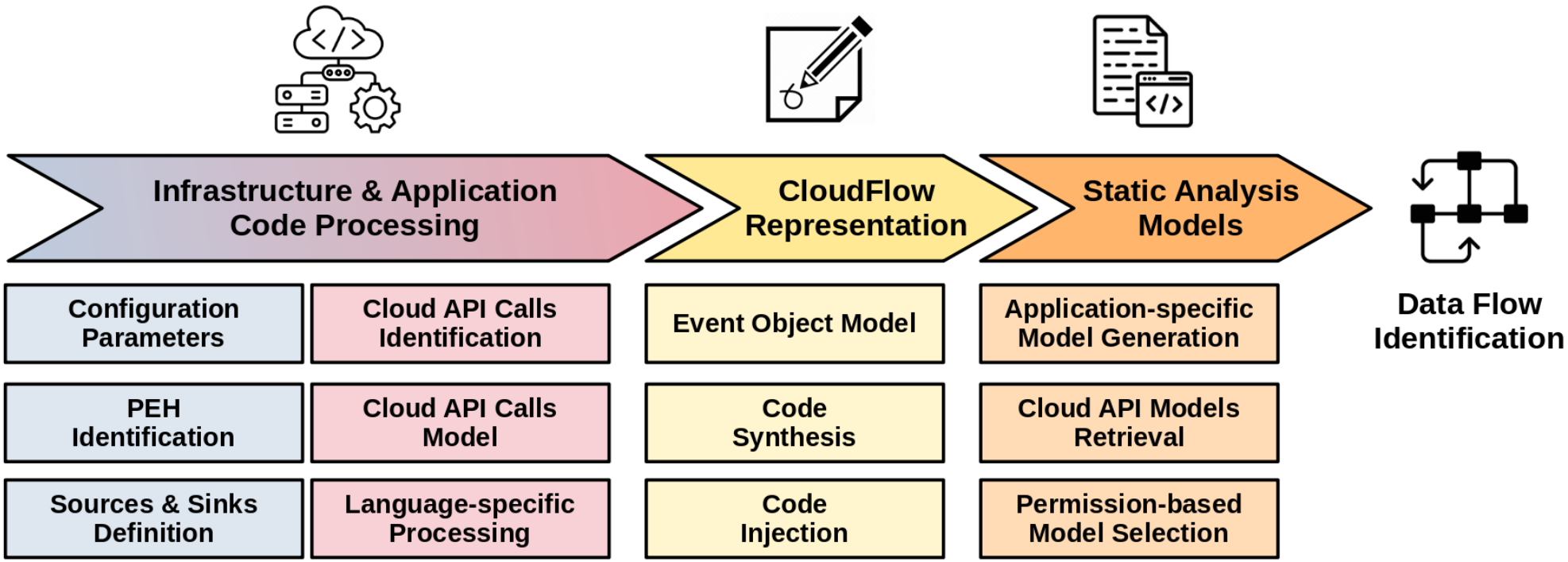
Configuration and management services

AWSomePy Characterization (2)

- **Cloud APIs**
  - Programmatic creation of buckets & tables
    - Resources cannot be checked before deployment
  - Use of invoke API to trigger handler execution
    - Added workflows not easily detectable via static analysis

s3 API	#	**dynamodb** API	#	**lambda** API	#
put_object	61	put_item	143	invoke	55
get_object	52	scan	64	add_permission	7
create_bucket	50	query	62	list_functions	3
upload_file	48	get_item	58	get_policy	3
download_file	24	update_item	57	get_function	2
list_objects_v2	22	create_table	41	list_tags	2
other	111	*other*	93	*other*	4

CloudFlow Analysis Pipeline



- **CloudFlow representation**
  - Application code instrumentation
  - Synchronous-like program execution flow

Microbenchmark-based Evaluation

- **CloudBench suite**
  - 40 AWS Python applications
  - 30 inter-procedural
  - 9 intra-procedural
  - 27 cases of code injection

Category	Amount
Inter-procedural	30
Intra-procedural	9
Simple Applications	1

- **Evaluation results**
  - CloudFlow passes 37 microbenchmarks out of 40 ✔
  - 1 false positive (CloudFlow limitation) ✖
  - 2 false negatives (underlying static analysis tool) ✖

Ch. 5

Real-world Application Analysis

- **Findings**
  - 104 real-world applications analysed

CloudFlow detects 205 data flows

Category	Total No.
Vulnerability	11
Potential Vulnerability	64
No Vulnerability	130

- **Vulnerabilities**

Unchecked CLI parameters

Data flow triggered by HTTP request

Vulnerability Type	Total No.
Code Injection via CLI	2
Code Injection via HTTP	5
Exception Traceback Leakage	4

Vulnerability Case Study

- **Exception traceback leakage**
  - Information about exception returned in HTTP response
  - Not directly exploitable, but should be used only for testing
  - Case of intra-procedural data flow
  - One case study for each category

Ch. 6

```
def event_process(missing_event) -> Response:
    try:
        # Processing and logging commands
    except Exception as e:
        return Response(str(e), status=500)
```

Publications

- **SESAME 2023**

AWSomePy: A Dataset and Characterization of Serverless Applications

Ch. 3

- **SANER 2024**

Towards Inter-service Data Flow Analysis of Serverless Applications

Ch. 4 Ch. 5

- **USENIX Security 2025**

CloudFlow: Identifying Security-sensitive Data Flows in Serverless Applications

Ch. 4 Ch. 5 Ch. 6

Conclusion

- **Key takeaways**

AWSomePy dataset characterization	→	**Workflows and resources difficult to inspect statically**
CloudFlow for detection of security-sensitive data flows	→	**Infrastructure analysis and synchronous representation**
CloudBench suite of security microbenchmarks	→	**CloudFlow passes 37 out of 40 microbenchmarks**
Analysis of real-world apps with CloudFlow	→	**Identified 11 vulnerabilities in 104 applications**

Final Remarks

- **Future work**

Higher number of services & APIs	Automated API docs analysis	Alternative static analysis tools

- **Recommendations**

Standardization	Design Patterns	Research Datasets